

GOVERNMENT DEGREE COLLEGE BEERWAH

PYTHON PROGRAMMING

Introduction to Python

6th Semester (BCA)

Introduction

release data: 20 FEB 1991

Open source language
#2 on TIOBE Index

Python is one of the most popular and widely used programming languages today.

Python is a **high-level, interpreted programming language** created with the goal of being easy to read and simple to use.

It supports multiple programming paradigms, including **object-oriented | procedural | and functional programming**.

Python is widely used for **Web Development | Data Science | Artificial Intelligence | Automation | Mobile Development** etc.

Origins and History of Python .

Creator: **Guido van Rossum** began developing Python in the late 1980s as a successor to the **ABC language**.

Inspiration: Guido was working on the **ABC language** at the **Centrum Wiskunde & Informatica (CWI)** in the **Netherlands**. While ABC was designed for teaching programming, it had limitations.

Guido wanted to create a language that combined the best features of ABC with the power and flexibility of languages like C.

Name: The name "**Python**" was inspired by the British comedy Show "**Monty Python's Flying Circus**", as Guido was a fan of their work.

The name was chosen to be **short, unique**, and **slightly mysterious**.

Origins and History of Python ..

First Release: Python **0.9.0** was released in **1991**, featuring exception handling, functions, and core data types.

Python 1.0: Released in **1994**, it included features like **lambda functions** and basic data structures.

Python 2.0: Released in **2000**, it introduced **list comprehensions, garbage collection, and Unicode support.**

Python 3.0: Released in **2008**, it was a major overhaul of the language and is not backward-compatible with Python 2.x. **Python 3.x emphasizes cleaner syntax and better functionality.**

End of Python 2.x: Python 2.x reached its end of life on **January 1, 2020**, and is no longer supported.

February 14, 2025, the latest stable release of Python is **Python 3.13.2**, which was released on **February 4, 2025**

Python Versions	Released Date
Python 1.0	January 1994
Python 1.5	December 31, 1997
Python 1.6	September 5, 2000
Python 2.0	October 16, 2000
Python 2.1	April 17, 2001
Python 2.2	December 21, 2001
Python 2.3	July 29, 2003
Python 2.4	November 30, 2004
Python 2.5	September 19, 2006
Python 2.6	October 1, 2008
Python 2.7	July 3, 2010
Python 3.0	December 3, 2008
Python 3.1	June 27, 2009
Python 3.2	February 20, 2011
Python 3.3	September 29, 2012
Python 3.4	March 16, 2014
Python 3.5	September 13, 2015
Python 3.6	December 23, 2016
Python 3.7	June 27, 2018
Python 3.8	October 14, 2019
Python 3.9	October 5, 2020
Python 3.10	October 4, 2021
Python 3.11	October 24, 2022
Python 3.12	October 2, 2023
Python 3.12.3	April 9, 2024
Python 3.13.2	February 4, 2025

Structure of a Python Program

Python supports multiple programming paradigms:

Procedural Programming – Step-by-step execution using functions

Functional Programming – Uses pure functions, immutability, and higher-order functions

Object-Oriented Programming (OOP) – Uses classes and objects for organization

Module-Based Programming – Divides the code into reusable modules

Interpreter shell

The **Python Interpreter Shell**, also known as the **REPL** (Read-Eval-Print Loop), is an **interactive environment** where you can **execute Python code line by line**.

It is a **powerful tool for testing** small code **snippets, debugging**, and **learning** Python.

The Python Interpreter Shell is a **command-line tool** that allows you to interact with Python in **real-time**.

It **reads** your **input**, **evaluates** it, **prints the result**, and loops back to wait for more input (hence the name REPL).

How to Access the Python Interpreter Shell

On Windows:

- ◆ *Open the Command Prompt (cmd).*
- ◆ *Type python or python3 and press Enter.*

On macOS/Linux:

- ◆ *Open the Terminal.*
- ◆ *Type python3 and press Enter.*

Exiting the Python Interpreter Shell

To exit the shell, type exit() or quit(), or press Ctrl + Z (Windows)

Ctrl + D (macOS/Linux).

```
>>> print("Hello, Rather!")  
Hello, Rather!
```

Indentation

Indentation is a fundamental aspect of Python syntax. Unlike many other programming languages that use braces `{}` or keywords like *begin* and *end* to define **blocks of code**,

Python uses **whitespace indentation** to group statements. This makes Python code **highly readable** and enforces a **consistent style**

- ◇ Indentation refers to the **spaces or tabs** at the **beginning** of a line of code.
- ◇ It is used to define **blocks of code** such as **loops, conditionals, functions, and classes**.
- ◇ The standard convention is to **use 4 spaces per indentation level** (as recommended by **PEP 8, Python's** style guide).

Why is Indentation Important in Python?

- ◇ **Readability:** Indentation makes the structure of the code visually clear.
- ◇ **Syntax Requirement:** Incorrect indentation will result in an *IndentationError*.
- ◇ **Logical Grouping:** Indentation defines the **scope of loops, conditionals, and functions**.

Comments

Comments are an essential part of any programming language, including Python. They are used to explain the purpose of the code, make it more readable, and provide context for other developers (or your future self). Comments are ignored by the Python interpreter and do not affect the execution of the program.

Single-Line Comments

Used for short explanations or notes.

Single-line comments start with the **#** symbol. Everything after **#** on that line is ignored by the Python interpreter.

Best practices:

- ❖ Write clear, concise, and meaningful comments.

Multi-Line Comments

Used for longer descriptions or documentation.

Python uses **multi-line strings** (enclosed in triple quotes **"""** or **"""**) to achieve the same effect.

Identifiers .

Identifiers are names used to identify *variables, functions, classes, modules*, or other *entities* in a program. They are essentially labels that allow programmers to refer to these entities within the code.

Rules for Identifiers:

◆ Allowed Characters:

- ◆ Identifiers can include *letters (a-z, A-Z), digits (0-9), and underscores (_)*.
- ◆ They must begin with a letter or an underscore (not a digit).

◆ Case Sensitivity:

- ◆ Identifiers are case-sensitive in most programming languages (e.g. *myVar* and *myvar* are different).

◆ Reserved Words:

- ◆ Identifiers cannot be the same as reserved keywords (e.g. *if, else, for, class* in python)

◆ Length:

- ◆ Most languages do not impose a strict limit on the length of identifiers, but it's good practice to keep them concise and meaningful.

Identifiers ..

Best Practices:

- ◆ Use descriptive names to make the code more readable (e.g., `total_sales` instead of `ts`).
- ◆ Follow naming conventions specific to the language (e.g., `snake_case` in Python, `camelCase` in JavaScript).
- ◆ Avoid using single-letter names unless they are obvious (e.g. *i* for a loop counter).

Examples of Valid Identifiers:

myVariable,
_privateVar,
total_count,
MAX_VALUE,
user123

Examples of Invalid Identifiers:

123user (starts with a digit),
my-var (hyphen not allowed),
class (reserved keyword)

Python: Uses *snake_case* for variables and functions, and *camelCase* for variables/methods and *PascalCase* for classes.

Keywords .

Keywords are reserved words in a programming language that have special meanings and are used to define the syntax and structure of the code.

They cannot be used as identifiers (e.g., variable names, function names) because they are part of the language's core functionality.

Each programming language has its own set of keywords.

Characteristics of Keywords:

- ◆ **Reserved:** Keywords are predefined and cannot be redefined or used for other purposes.
- ◆ **Case-Sensitive:** In most languages, keywords are case-sensitive (e.g. **if** is a keyword but **IF** is not).
- ◆ **Language-Specific:** Different programming languages have different sets of keywords.

Keywords ..

List of Python Keywords (as of Python 3.x)

◆ Control Flow:

- ◆ *if, else, elif, for, while, break, continue, pass*

◆ Functions and Classes:

- ◆ *def, return, lambda*

◆ Logical Operators:

- ◆ *and, or, not, is, in*

◆ Exception Handling:

- ◆ *try, except, finally, raise, assert*

◆ Imports and Modules:

- ◆ *import, from, as*

◆ Variables and Scope

- ◆ *global, nonlocal*

◆ Boolean Values:

- ◆ *True, False*

◆ Null Value:

- ◆ *None*

◆ Asynchronous Programming:

- ◆ *async, await*

◆ Others

- ◆ *del, with, yield*

Literals

In Python, **literals** are raw data or values that are directly assigned to variables or used in expressions. They represent fixed values in the source code and do not require computation or evaluation.

Python supports various types of literals, including

Numeric literals: Represent numbers (*integers, floats, and complex numbers*).

String literals: Represent sequences of *characters* enclosed in quotes ' " ''' """

Boolean literals: Represent truth values (*true | false*).

None Literal: Represents the absence of a value or a null value.

List Literals: Represent ordered, mutable collections of items enclosed in square brackets `[]`.

Tuple Literals: Represent ordered, immutable collections of items enclosed in parentheses `()`.

Dictionary Literals: Represent key-value pairs enclosed in curly braces `{}`.

Set Literals: Represent unordered collections of unique items enclosed in curly braces `{}`.

Bytes Literals: Represent sequences of bytes (raw binary data) prefixed with *b*.

F-Strings (Formatted String Literals): Allow embedding expressions inside string literals using `{}`. Prefixed with *f*.

Variables .

Variables are used to store values that can be used and manipulated throughout a program.

A variable is essentially a named location in memory where a value is stored.

Variables act as containers for data, and they allow you to refer to the data by a name rather than the actual value.

Assigning Values to Variables

- ◇ To store a value in a variable, you use the **assignment operator (=)**.

variable_name = value

Rules for Variable Names:

- ◇ Variable names must start with a letter (a-z, A-Z) or an underscore (_).
- ◇ They can contain letters, digits (0-9), and underscores.
- ◇ Variable names are case-sensitive (*myVar* and *myvar* are different).
- ◇ Avoid using Python keywords (e.g. *if*, *else*, *for*) as variable names.

Dynamic Typing:

- ◇ Python is a dynamically typed language, meaning *you don't need to declare the type of a variable*. The type is inferred from the value assigned to it.

Variables ..

Multiple Assignments:

- ◇ You can assign values to multiple variables in a single line.

Example `(a, b, c = 1, 2, 3 # a = 1, b = 2, c = 3)`

Constants:

- ◇ In Python, there is **no strict concept of constants** (values that cannot change). However, by convention, **constants** are written in **uppercase** to indicate that they should not be modified.
- ◇ Example `(PI = 3.14)`

Reassigning Variables:

- ◇ You can change the value of a variable by reassigning it.
- ◇ Example `(x = 10`

`print(x) # Output: 10`

`x = 20 # Reassign x to 20`

`print(x) # Output: 20)`

Variables ...

Storing Different Data Types:

- ◇ Variables can store values of any data type, including numbers, strings, lists, dictionaries, etc.
- ◇ Example

```
# Integer  
age = 25  
  
# Float  
height = 5.9  
  
# String  
name = "Alice"
```

Checking Variable Types:

- ◇ You can use the *type()* function to check the data type of a variable.
- ◇ Example

```
x = 10  
  
print(type(x)) # Output: <class 'int'>
```

Deleting Variables:

- ◇ You can delete a variable using the *del* keyword. This removes the variable from memory.
- ◇ Example

```
x = 10  
  
print(x) # Output: 10  
  
del x  
  
print(x) # Error: NameError: name 'x' is not defined
```

Basic operators

◆ Arithmetic operator

- ◆ + # Addition
- ◆ - # Subtraction
- ◆ * # Multiplication
- ◆ / # Division
- ◆ % # Modulus (remainder)
- ◆ ** # Exponentiation (power)
- ◆ // # Floor division (quotient without remainder)

◆ Relational operator

- ◆ == : Equal to
- ◆ != : Not equal to
- ◆ > : Greater than
- ◆ < : Less than
- ◆ >= : Greater than or equal to
- ◆ <= : Less than or equal to

◆ Logical (Boolean) operator

- ◆ and : True if both conditions are true.
- ◆ or : True if at least one condition is true.
- ◆ not : Reverses the result (True becomes False, and vice versa)

◆ Identity Operators

- ◆ is # True if both variables point to the same object
- ◆ is not # True if both variables do not point to the same object

◆ Assignment Operator

- ◆ = # Assign
- ◆ += # Add and assign
- ◆ -= # Subtract and assign
- ◆ *= # Multiply and assign
- ◆ /= # Divide and assign
- ◆ %= # Modulus and assign
- ◆ **= # Exponent and assign
- ◆ //= # Floor divide and assign

◆ Bit wise operator

- ◆ & : Bitwise AND
- ◆ | : Bitwise OR
- ◆ ^ : Bitwise XOR
- ◆ ~ : Bitwise NOT (inverts bits)
- ◆ << : Left shift (shifts bits to the left)
- ◆ >> : Right shift (shifts bits to the right)

◆ Membership Operators

- ◆ in # True if value is found in the sequence
- ◆ not in # True if value is not found in the sequence

Data Types

Category	Data Type	Description
Numeric	int	Integer numbers (e.g., 10, -5)
	float	Decimal numbers (e.g., 3.14, -10.5)
	complex	Complex numbers (e.g., 2 + 3j)
Sequence	string	Strings (e.g., "Hello", 'Python')
	list	Ordered, mutable collections (e.g., [1, 2, 3])
	tuple	Ordered, immutable collections (e.g., (1, 2, 3))
Mapping	dictionary	Key-value pairs (e.g., {"name": "Alice"})
Set	set	Unordered, unique collections (e.g., {1, 2, 3})
	frozenset	Immutable, unique collections (e.g., frozenset({1, 2, 3}))
Boolean	bool	Truth values (True or False)
None	None	Represents absence of a value
Binary	bytes	Immutable binary data (e.g., b"hello")
	bytearray	Mutable binary data (e.g., bytearray(b"hello"))
	memoryview	Memory view of binary data

Creating Python Programs

Creating Python programs involves writing code in a **.py** file and executing it using the Python interpreter.

A Python program is a text file with a **.py** extension that contains Python code.

Writing a Python Program:

Steps:

- ◇ Open a text editor or an Integrated Development Environment (IDE) like:
 - ◇ **Text Editors:** *Notepad* (Windows), *TextEdit* (Mac), or *VS Code*, *Sublime Text*, *Atom*.
 - ◇ **IDEs:** *PyCharm*, *Thonny*, *Jupyter Notebook*, or *Spyder*.
- ◇ Write your Python code in the file. eg. `# hello.pyprint("Hello, World!")`
- ◇ Save the file with a .py extension. eg. `hello.py`
- ◇ You can run a Python program using the Python interpreter.
- ◇ Open a terminal or command prompt.
- ◇ Navigate to the directory where your .py file is saved and type the following command
`python hello.py` # here *hello.py* is the program name

Input and Output Statements .

In Python, **input** and **output** (I/O) operations are used to interact with the user and display results. Python provides built-in functions like *input()* and *print()* for handling I/O operations.

Output Statements:

- ◇ Output statements are used to display data to the user. The most common way to output data in Python is using the *print()* function.

Syntax

```
print(value1)
```

```
print(value1, value2, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

Parameters:

value1, value2, ...: Values to be printed (can be strings, numbers, variables, etc.).

sep: Separator between values (default is a space ' ').

end: Character to be printed at the end (default is a newline '\n').

file: File object where output is written (default is sys.stdout, which is the console).

flush: Whether to forcibly flush the stream (default is False).

Input and Output Statements ..

Input Statements:

- ◇ Input statements are used to take input from the user. The *input()* function is used to read input from the console.

Syntax

input(prompt)

variable = input("enter any value") *# any value considered as string*

variable = input() *#any value*

variable = int(input("enter an integer value")) *# integer values*

Parameters:

prompt: A string message displayed to the user (optional).

The *input()* function reads a line of text from the user.

It always returns the input as a **string** If you need a different data type, you must convert it explicitly.

Control statements .

- ◆ **Branching**

Branching allows you to execute different blocks of code based on conditions.

if, if-else, if-elif-else

- ◆ **Looping**

Looping allows you to repeatedly execute a block of code.

for, while

- ◆ **range function**

The range() function generates a sequence of numbers, commonly used in for loops.

- ◆ **exit function**

The exit() function is used to terminate the program.

- ◆ **break**

Exits the loop immediately.

- ◆ **continue**

Skips the rest of the code in the current iteration and moves to the next iteration.

- ◆ **pass**

Used as a placeholder for future code. It does nothing.

Control statements ..

Branching

In Python, **branching** refers to the use of **conditional statements** to control the flow of a program based on certain conditions.

Branching allows your program to make decisions and execute different blocks of code depending on whether a condition is true or false.

The primary conditional statements used for branching in Python are:

- ◇ *if* Executes a block of code if a condition is true
- ◇ *if-else* Executes one block if true and another if false.
- ◇ *if-elif-else* Checks multiple conditions and executes the first true block.
- ◇ *Nested if* Handles complex conditions by nesting if statements.
- ◇ *Ternary operator* A shorthand for simple if-else conditions

Control statements ...

Looping

In Python, **looping** is used to repeatedly execute a block of code. Loops are essential for automating repetitive tasks and iterating over sequences like lists, tuples, strings, and ranges.

Python provides two main types of loops:

for loop

used to iterate over a sequence (e.g., list, tuple, string, range) and execute a block of code for each item in the sequence. It is one of the most commonly used loops in Python due to its simplicity and versatility.

while loop

used to repeatedly execute a block of code as long as a specified condition is true. It is useful when you don't know in advance how many times the loop needs to run. The loop continues until the condition becomes false.

Control statements

Syntax of for loop

for item in sequence:

Code to execute for each item

item: A variable that takes the value of each element in the sequence.

sequence: The collection of items to iterate over (e.g., list, tuple, string, range).

Syntax of while loop

while condition:

Code to execute while condition is true

condition: A Boolean expression. The loop continues as long as this condition evaluates to *true*.

The loop stops when the condition becomes *false*.

Control statements

range()

The *range()* function in Python is used to generate a sequence of numbers. It is commonly used in loops (e.g. *for* loop) to iterate over a sequence of numbers. The *range()* function is efficient because it generates numbers on the fly without storing the entire sequence in memory.

Syntax

range(stop) *Stop*: The ending value of the sequence (exclusive). This parameter is required.

range(start, stop) *Start*: The starting value of the sequence (inclusive). Default is 0.

range(start, stop, step) *Step*: The difference between each number in the sequence. Default is 1.

Key Features

- ◆ **Generates a Sequence of Numbers:**
 - ◆ The sequence starts from start and ends at stop -1.
 - ◆ The step parameter controls the increment between numbers.
- ◆ **Memory Efficient:**
 - ◆ *range()* does not store the entire sequence in memory. It generates numbers on the fly.

exit()

The *exit()* function is used to terminate the execution of a program. It is part of the *sys* module but can also be used directly without importing the module. When called, it stops the program and returns an exit status to the operating system.

Syntax

exit([status]) *status* : (Optional) An integer or string representing the exit status.

if status is integer

0 indicates successful termination. | Non-zero values indicate an error or abnormal termination.

if status is a string it is printed to the console before exiting.

Control statements

break | continue | pass

are control flow statements used to alter the behavior of loops and conditional statements.

break Statement

- ✧ The *break* statement is used to exit a loop immediately. When encountered, it terminates the loop and transfers control to the statement following the loop.

Use Case:

- ✧ Exiting a loop when a specific condition is met.

continue Statement

- ✧ The continue statement is used to skip the rest of the code in the current iteration of a loop and move to the next iteration.

Use Case:

- ✧ Skipping specific iterations based on a condition.

pass Statement

- ✧ The pass statement is a placeholder that does nothing. It is used when a statement is syntactically required but no action is needed.

Use Case:

- ✧ Placeholder for future code.
- ✧ Avoiding syntax errors in empty loops or conditional blocks.